

Apache y Nginx en Docker

Clase de **pura práctica**: primero cerró los entregables del avance de proyecto, y luego se sentó a construir con el grupo dos formas de publicar el minisito de nuevo ingreso en un contenedor — un `docker run` con todos sus flags, y la ruta que sí queda en el repo: un `Dockerfile`. El resto de la clase fue laboratorio: montar el mismo sitio en Apache y en Nginx, cada uno en su puerto.

La página sigue las **dos rutas de Docker** con un riel de color: **ámbar** **punteado** el `docker run` — rápido, pero no queda guardado en ningún lado — y **verde sólido** el `Dockerfile`, la que sí se versiona. Los servidores de la práctica llevan su propio color: **APACHE** **NGINX**.

Los botones con ? explican los términos que dio por sabidos. Hay seis ejercicios intercalados.

Lo que salió de esta clase

00:00

Avance de proyecto: diccionario de datos, modelo E-R, script SQL y su representación relacional. Al menos **5 tablas** sin contar gestión de usuarios, y la base en al menos **3FN**.

17:00

Práctica: publicar el minisito en un **APACHE** (puerto **8080**) y en un **NGINX** (puerto **8081**). Es el trabajo del resto de la clase.

31:45

Pendiente administrativo: "asuman que todo se entrega el 11 de septiembre" — ~15 actividades subidas a la plataforma, varias son solo lecturas.

confirmado

La **réplica del minisito de nuevo ingreso** (tarea de la clase 2) ya quedó entregada por el grupo — con ella se trabajó toda la sesión.

i

Nota sobre la fecha de esta clase: se infiere miércoles 9 de septiembre por el patrón lunes/miércoles del curso (la clase 5 fue el lunes 7). El propio profesor dice en esta sesión "todavía tenemos como una semanita" hablando del 11 de septiembre, lo cual encajaría mejor con una fecha unos días antes del 9. Vale la pena confirmarla contra tu propio calendario antes de fiarte de ella para otra cosa.

BLOQUE 0 · MINUTO 00:00 A 05:30

Lo que ya se debe entregar del proyecto

Antes de tocar Docker, cerró la lista de entregables del avance del primer parcial. El proyecto sigue siendo libre — equipos de hasta cuatro, problema a elección — pero estos cuatro documentos ya no son opcionales.

"Es libre, hasta ahí. ¿Vamos bien o nos regresamos? Entonces, ¿qué vamos a entregar hasta ahorita?"

Cuatro piezas, en el orden en que las nombró — aunque él mismo aclaró que **no es el orden en que se construyen**: el script SQL y la base ya debieron existir antes de poder sacar el diagrama entidad-relación con una herramienta.

1. **Portada.**
2. **Diccionario de datos:** la tabla con atributo, tipo, restricciones (numérico, autogenerado, llave primaria, no nulo...) y una descripción en prosa de qué representa cada campo.
3. **Modelo entidad-relación (E-R).**
4. **Script de creación de la base de datos** (el archivo `.sql`), del cual —con ingeniería inversa en el IDE que uses— sale el E-R y la representación relacional casi solos.

— ¿Qué hace exactamente la "reingeniería" que mencionó?

Es **ingeniería inversa**: en vez de dibujar el diagrama y de ahí generar el script SQL (el camino "de adelante hacia atrás"), le das a la herramienta una base de datos ya creada y ella dibuja el diagrama entidad-relación leyendo las tablas, columnas y llaves foráneas que encuentra. MySQL Workbench, DBeaver y Navicat lo hacen todos con un asistente de "Reverse Engineer" o similar.

Dos reglas de diseño, ambas ya vistas en Bases de Datos pero que insistió en repetir:

REGLA	DETALLE
≥ 5 tablas	Sin contar el módulo de gestión de usuarios. Ese módulo "va de cajón" — usuarios, roles y la tabla de unión entre ambos son tres tablas más , aparte.
3FN	Tercera Forma Normal, como mínimo. "Ya pasaron por bases de datos" — no lo explicó de nuevo, lo dio por hecho.

— ¿Qué exige la Tercera Forma Normal (3FN), en corto?

Que cada columna dependa **solo** de la llave primaria de su tabla, y de nada más. Si una columna en realidad describe a otra columna que no es la llave (por ejemplo, guardar el nombre de una carrera dentro de la tabla de alumnos en vez de solo su `id_carrera`),

esa columna se va a su propia tabla. Es la regla que evita que un mismo dato se repita en muchas filas y se desincronice.

EJERCICIO 1 · CUENTA LAS TABLAS

¿Cuántas tablas, según qué incluyas?

Tres preguntas sobre las reglas que acaba de poner. Contesta las tres y dale a **Revisar**.

¿Cuántas tablas como mínimo debe de tener tu diseño, sin contar gestión de usuarios?

☐ 3☐ 5☐ 10

Si **SÍ** incluyes gestión de usuarios (con roles), ¿cuántas tablas se suman a esas cinco?

☐ Una☐ Dos☐ Tres

¿En qué forma normal debe estar, como mínimo, tu base de datos?

☐ 1FN☐ 2FN☐ 3FN

BLOQUE 1 · MINUTO 05:30 A ~11:00

Ruta 1 — `docker run`, flag por flag

Retomó la presentación de contenedores donde se había quedado, y armó en el pizarrón el comando completo para publicar el minisitio sin escribir un solo archivo de configuración.

`docker run` · NO QUEDA GUARDADO EN NINGÚN LADO

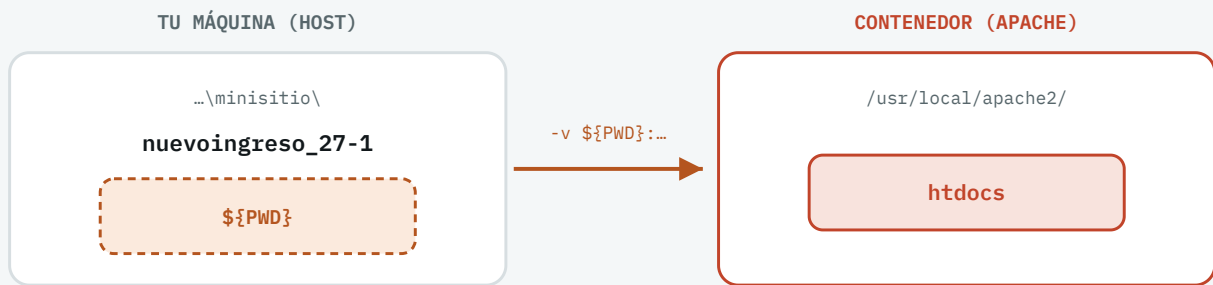
Partió de una estructura de carpetas concreta —la suya, en su propia máquina— para que quedara claro de dónde sale cada pieza del comando:

Documentos\escom\2027-1\escom\back-front\contenedores\minisitio\nuevoingreso_27-1 . Ahí adentro, con la terminal parada en esa carpeta, el comando fue:

```
docker run --name escom_ni -d -it -p 3000:80 httpd:latest -v ${PWD}:/usr/local/apache2,
```

*"El **ni** es de 'nuevo ingreso'. Queremos que esta cosa, si en dado caso llega a funcionar —esperemos que sí—, escuche peticiones para mí en*

el puerto 3000. Que internamente esta cosa va a estar trabajando en el puerto 80, eso es otra cosa."



De los dos puntos hacia la izquierda: dónde está lo tuyo en el host. De los dos puntos hacia la derecha: dónde lo espera Apache — `htdocs` es, por convención en la familia Debian/Ubuntu, el directorio de trabajo del servidor.

La pregunta clave de este bloque fue de dónde sale `$PWD` :

Alumno: "Mostrar el directorio donde te encuentras." **Profesor:**

"Mostrar el directorio actual. Funciona igual: o sea, estoy trabajando aquí. Entonces esto funciona para sistemas Unix, específicamente para Unix: `$PWD`."

— ¿Qué hace `pwd` y por qué aparece entre llaves como `$PWD` ?

`pwd` ("print working directory") es el comando de Unix/Linux que imprime la ruta de la carpeta donde está parada tu terminal ahora mismo. `$PWD` es la variable de entorno equivalente: en vez de escribir la ruta completa a mano dentro del comando `docker run`, dejas que el shell la sustituya sola por donde sea que estés parado. Así el mismo comando le sirve a cualquiera, sin importar dónde tenga clonada la carpeta.

El profesor mismo se trabó buscando el nombre de `{ }` —dijo "llavecitas", "barritas", "corchetes" antes de que un alumno lo corrigiera a "llaves"— y no llegó a aclarar del todo el equivalente exacto en PowerShell. Lo seguro: en Windows **no** se usa `$PWD` tal cual como en Bash; conviene probarlo en tu propia terminal antes de copiar el comando literal.

Resultado esperado: abrir `localhost:3000` en el navegador y ver el minisitio, servido por el Apache que vive dentro del contenedor.

EJERCICIO 2 · DECODIFICADOR DE FLAGS

¿Qué hace cada parte del `docker run`?

Sobre este mismo comando: `docker run --name escom_ni -d -it -p 3000:80 httpd:latest -v ${PWD}:/usr/local/apache2/htdocs`.

Empareja cada pieza con lo que hace.

`--name escom_ni`

Nombre del contenedor

Nombre de la imagen

Puerto por el que escucha

`-d`

Modo interactivo

Modo desatendido (segundo plano)

Copia el directorio actual

`-it`

Permite conectarte por terminal a un contenedor en ejecución

Instala herramientas dentro del contenedor

Indica el tag (versión) de la imagen

`-p 3000:80`

El contenedor escucha en 3000, por dentro trabaja en 80

El contenedor escucha en 80, por dentro trabaja en 3000

Son el mismo puerto, solo cambia el nombre

`-v ${PWD}:/usr/local/apache2/htdocs`

Copia el contenido una sola vez, al crear el contenedor

Monta (asocia en vivo) tu carpeta actual con la carpeta htdocs

Cambia el puerto por el que responde Apache

Revisar

Reiniciar

BLOQUE 2 · MINUTO ~11:00 A 17:00

Ruta 2 — el `Dockerfile`, la que sí se guarda

El mismo resultado, pero esta vez como archivo de configuración dentro del proyecto — reproducible, versionable, y sin tener que reescribir el

comando cada vez.

DOCKERFILE • VIVE EN EL REPO

"¿No lo quiero echar a andar así? Está bien. Vamos con un Dockerfile."

La regla del nombre del archivo, tal cual la dijo: **sin extensión y con «D» mayúscula** — `Dockerfile`, no `dockerfile.txt` ni `Dockerfile.txt`.

```
FROM httpd

COPY . /usr/local/apache2/htdocs/

EXPOSE 80
```

— ¿Qué hace cada línea del Dockerfile?

- `FROM httpd` — parte de la imagen base `httpd`. Sin poner una versión después de los dos puntos, Docker toma la **última estable** disponible en el Docker Hub.
- `COPY . /usr/local/apache2/htdocs/` — copia el contenido de la carpeta donde está el Dockerfile (el punto, "el directorio actual") hacia adentro del contenedor, al directorio de trabajo de Apache.
- `EXPOSE 80` — documenta que el contenedor va a escuchar peticiones por el puerto 80 por dentro. No abre el puerto hacia el host por sí solo; eso lo hace el `-p` del `docker run`.

— ¿Qué es el Docker Hub?

Es el repositorio público de imágenes de Docker — el equivalente a lo que es GitHub para el código, pero para imágenes ya armadas. `httpd` y `nginx` son imágenes oficiales que cualquiera puede descargar desde ahí con un `FROM` o un `docker pull`.

Con el Dockerfile ya dentro de la carpeta del proyecto (junto al `index.html`, la carpeta `media`, `css`, etc.), toca construir la imagen:

```
docker build -t mi_imagen .
```

`-t` (o `--tag`) es el nombre que le pones a la imagen — puede llevar tu usuario delante, como `davis/mi_imagen:latest`. El punto al final **no es decorativo**: es el directorio donde Docker debe buscar el Dockerfile, o sea, el directorio actual.

Un `docker image ls` confirma que la imagen ya existe. Y para echarla a andar:

```
docker run --name escom_ni -d -it -p 3500:80 mi_imagen
```

Mismo resultado que con la ruta 1, solo que ahora por `localhost:3500`, y con la ventaja de que el Dockerfile queda en el repo — cualquiera puede reconstruir la imagen sin que tú le dictes el comando.

EJERCICIO 3 · ARMA LA RUTA 2 EN ORDEN

Del Dockerfile al contenedor corriendo

Cinco piezas: tres líneas del Dockerfile y dos comandos de terminal, revueltas. Ve armando la secuencia completa — construir la imagen y correrla no sirve si el Dockerfile está mal.

Aquí se va armando la ruta...

```
docker run --name escom_ni -d -it -p 3500:80 mi_imagen
```

EXPOSE 80

```
docker build -t mi_imagen .
```

FROM httpd

COPY . /usr/local/apache2/htdocs/

Reiniciar

BLOQUE 3 · MINUTO 17:00 A 31:40

La práctica: el mismo sitio, dos servidores

Con las dos rutas ya explicadas, soltó al grupo a trabajar. El resto de la clase la pasó circulando entre las máquinas resolviendo dudas puntuales.

PRÁCTICA DE LA CLASE

minuto 17:00

Publica el minisitio en **APACHE** y en **NGINX**

"Tienen lo que resta de la clase: móntenlo en un Apache Web Server.

Entreténganse. Es más: móntenlo en los dos, en un Apache y en una cosa rara que se llama Nginx. Lo divertido es: este pónganlo en el puerto 8080 y este pónganlo en el puerto 8081."

Es el mismo minisitio de nuevo ingreso de la clase 2, con Dockerfile ya hecho para el puerto 3500 (tarea 2). Ahora toca duplicarlo: una imagen basada en `httpd` expuesta en **8080**, y otra basada en `nginx` expuesta en **8081**.

```
docker run --name escom_apache -d -it -p 8080:80 mi_imagen_apache
docker run --name escom_nginx -d -it -p 8081:80 mi_imagen_nginx
```

La única pieza nueva es el Dockerfile de Nginx, porque su carpeta de trabajo por defecto **no es** `htdocs` como en Apache:

```
FROM nginx

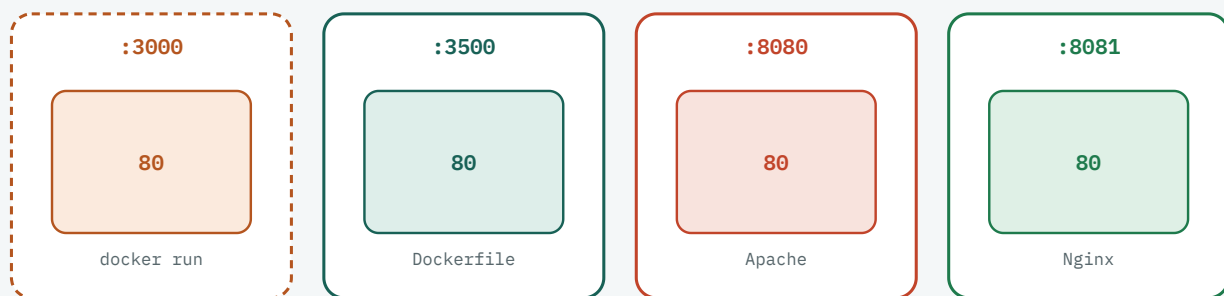
COPY . /usr/share/nginx/html/

EXPOSE 80
```

– ¿Qué es Nginx, y en qué se distingue de Apache aquí?

Nginx (se pronuncia "engine-x") es otro servidor web, como Apache — recibe peticiones HTTP y devuelve archivos o respuestas. Para este ejercicio la diferencia práctica que importa es una sola: la carpeta donde cada uno espera encontrar tus archivos estáticos. En Apache es `/usr/local/apache2/htdocs/`; en la imagen oficial de Nginx es `/usr/share/nginx/html/`. Cambiar solo esa ruta en el `COPY` es, para este ejercicio, la única diferencia real entre los dos Dockerfiles.

UN PUERTO INTERNO, CUATRO PUERTAS DE ENTRADA



Los cuatro contenedores de la clase trabajan por dentro en el puerto 80 — es el puerto por defecto de cualquier servidor web. Lo único que cambia entre uno y otro es el puerto del **host**, el de la izquierda de los dos puntos en `-p`.

Antes de arrancar: el inventario del laboratorio

Mientras el grupo se instalaba, aproveché para pasar lista de lo que cada máquina del laboratorio "se supone" que ya trae — y pidió confirmarlo, no darlo por hecho:

- **Docker** — indispensable para todo lo de hoy.
- **Java** — "en algún momento vamos a jugar con esto", averiguar qué versión.
- **Visual Studio Code**

● DBeaver o MySQL

● IntelliJ

● Insomnia (o Postman)

Si tu máquina del laboratorio no trae algo de esta lista, la alternativa que dio fue simple: trae tu propio equipo.

DIGRESIÓN · MINUTO 31:45 A 36:00

"Asuman que todo se entrega el 11 de septiembre"

"De las 20,000 actividades que su compañero me imagino ya les compartió... asuman que todo se entrega el 11 de septiembre. [...] Ya si para el 11 de septiembre, digamos que ya estamos como por el 9 y les dejo una actividad, no va a ser para el 11, va a ser para una semana después."

Un alumno contó ~18 actividades subidas a la plataforma, otro corrigió a ~15. El profesor cerró el conteo sin pelear el número exacto: "es un montón", pero aclaró que **no todas son entregables calificados** — varias son solo lecturas que espera que se hagan por cuenta propia, sin revisión.

- i Este "11 de septiembre" es una **fecha general**, dicha de pasada para el conjunto de actividades de la plataforma — no quedó claro si aplica tal cual al avance de proyecto del Bloque 0, que en la lista oficial del curso seguía sin fecha propia hasta esta clase. Se registra con esa reserva en la agenda del semestre.

BLOQUE 4 · MINUTO 36:00 A 45:10

Varios servicios a la vez: Docker Compose

Cuando el proyecto necesita más de un contenedor —Apache más Tomcat, o Apache más Nginx— ya no conviene escribir un `docker run` por cada uno a mano.

"Si quieren montar varios servicios, móntenlos en un archivito de este estilo: `docker-compose.yml`."

La idea es reciclar exactamente la misma estructura que ya tienen: un Dockerfile por servicio, cada uno en su propia carpeta, y el archivo `docker-compose.yml` apuntando a cada una.

```
services:
  apache:
    build: ./apache
    ports:
      - "8080:80"
  nginx:
    build: ./nginx
    ports:
      - "8081:80"
```

– ¿Qué es Docker Compose y en qué se distingue de un docker run suelto?

Docker Compose es una herramienta que lee un archivo YAML (`docker-compose.yml`) donde describes uno o varios servicios —cada uno con su imagen o Dockerfile, sus puertos, sus volúmenes— y los levanta o los apaga todos juntos con un solo comando. Es la forma ordenada de reemplazar una fila de comandos `docker run` escritos a mano, uno por contenedor.

```
docker compose up
docker compose down
```

`docker compose up` construye (si hace falta) y levanta todos los servicios del archivo. `docker compose down` los apaga y los quita.

"Y san se acabó. [...] Y vámonos."

– ¿Por qué la indentación de YAML importa tanto?

YAML usa la sangría (espacios, nunca tabulaciones) para representar de qué depende cada línea — no llaves ni corchetes como JSON. Dos espacios de más o de menos cambian a qué servicio pertenece una clave, o la rompen directamente. Es justo el tipo de error que no truena con un mensaje claro: el archivo "se ve bien" pero Compose lo interpreta distinto a como lo pensaste.

Una alumna preguntó por la sintaxis en Windows. La respuesta se quedó a medias —el profesor dudó entre "llavecitas", "barritas" y "corchetes" antes de que el grupo lo corrigiera a "llaves"— pero el punto de fondo sí quedó claro: en PowerShell el comando casi se copia y pega igual, con Docker instalado, y el resultado esperado es el mismo: se conecta al Docker Hub, descarga la imagen, toma tu carpeta actual, crea el contenedor con el nombre que le diste, y expone el puerto que pediste.

EJERCICIO 4 · DOCKER COMPOSE

De varios `docker run` a un solo archivo

Tres preguntas sobre cómo se comporta Compose frente a levantar cada contenedor a mano.

En `docker-compose.yml`, cada servicio (apache, tomcat, nginx...) apunta a...

Su propio Dockerfile, con build: `./carpeta`

Un `docker run` escrito a mano por servicio

Un volumen compartido obligatorio entre todos

Para levantar todos los servicios del archivo de una sola vez:

`docker compose up`

`docker run --all`

`docker build --compose`

Para apagarlos y quitarlos:

`docker compose stop`, nada más

`docker compose down`

`docker kill *`

Revisar

Reiniciar

BLOQUE 5 · MINUTO 45:15 A 49:08

¿Esto ya son microservicios?

Cerró la clase respondiendo una pregunta que alguien lanzó mientras trabajaba: si estamos jugando con servidores dentro de contenedores, ¿ya estamos haciendo microservicios?

"El cuestionamiento del compañero se me hizo interesante: ¿Estamos jugando con servidores?' La respuesta es sí. [...] Solo que en ese contenedor, que es ese espacio grandote, esa cajota que ven en los trenes y en los barcos así bien bonito —no sé, allá por Azcapotzalco hay un montón de contenedores—, ahí están empaquetando el servidor web, que es el Apache."

Estirando la misma metáfora: en otra "cajota" va un Nginx; en otra, un proyecto en Node; en otra, un proyecto en Spring —que además trae su propio servidor adentro, uno minimalista llamado **Tomcat embebido**—; en otra, un Angular. Y esas cajas, tarde o temprano, **tienen que platicar entre sí**.

– ¿Qué es un PaaS, y por qué Render puede correr Java sin "tener soporte para Java"?

PaaS (Platform as a Service) es una nube que te da un lugar donde correr tu aplicación sin que tú administres el servidor físico ni el sistema operativo por debajo — solo subes tu código o tu imagen y la nube se encarga del resto. Algunas PaaS dan soporte nativo a ciertos lenguajes (subes tu `.jar` y ya). Render, según el profesor, no tiene ese soporte nativo para Java — pero si tu proyecto ya vive dentro de una imagen Docker, a Render no le importa qué hay adentro: solo corre el contenedor. Empaquetar en Docker es, en ese sentido, un pasaporte universal.

Y sobre la duda de un compañero al final de la clase, mientras levantaba su propia landing page en otro puerto: si dos contenedores usan la misma imagen de servidor web y ambos escuchan por dentro en el 80, lo que se **enmascara** —lo único que cambia entre uno y otro— es el puerto del host. Es exactamente lo que pasó con Apache en 8080 y Nginx en 8081 en esta misma clase.

Contenedores, Tomcat embebido y PaaS

Cuatro preguntas sobre la reflexión final. La última es la misma duda que resolvió el profesor en vivo con un alumno.

¿Qué corre, en el fondo, dentro del contenedor de Apache de la práctica de hoy?

Un servidor web completo, aislado del resto del sistema

Solo el archivo index.html, sin servidor detrás

Una máquina virtual completa, con su propio kernel

Un proyecto en Spring, ¿trae su propio servidor por dentro?

No, siempre hay que instalar un Tomcat aparte

Sí: un Tomcat minimalista llamado 'Tomcat embebido'

No, Spring no usa Tomcat para nada

Render no tiene soporte nativo para Java. ¿Por qué de todos modos puedes desplegar ahí un proyecto Java?

Porque sí tiene soporte para Docker, y adentro puedes meter lo que quieras

Porque Render traduce Java a JavaScript solo

No se puede: hay que usar otro proveedor

Quieres correr dos sitios con la misma imagen de Apache al mismo tiempo, sin que truenen entre sí. ¿Qué cambias de un `docker run` a otro?

El puerto del host (-p), dejando el interno en 80 los dos

El puerto interno del contenedor, dejando el host en 80 los dos

Nada — Docker los acomoda solo

Revisar

Reiniciar

CIERRE

Checklist de la clase 6

Se guarda en tu navegador. Nada se sube a ningún lado ni se comparte con nadie más.

☐ **Práctica · publicar el minisitio en Apache (puerto 8080)**

Dockerfile con FROM httpd, COPY hacia /usr/local/apache2/htdocs/, EXPOSE 80, y docker run con -p 8080:80.

☐ **Práctica · publicar el mismo minisitio en Nginx (puerto 8081)**

Mismo Dockerfile, dos cambios: FROM nginx y COPY hacia /usr/share/nginx/html/. docker run con -p 8081:80.

☐ **Avance de proyecto · diccionario, E-R, script SQL y relacional**

Al menos 5 tablas sin gestión de usuarios (8 si la incluyes), base en 3FN. Fecha tentativa: 11 de septiembre — sin confirmar del todo para esta entrega en particular.

☐ **Pendiente administrativo · revisar tus actividades en la plataforma**

~15 actividades subidas, varias son solo lecturas sin entrega. El profesor las dio todas por vencidas el 11 de septiembre.